# 开源贡献指南

# 开源贡献指南（v3：双主社区信任线，与 P1/P2 步骤耦合）

> 2026-10-05 v3 重构：本版依据 04/05 的 v3 重构与 2026-10-05 的社区现勘重写，取代 v2（2026-10-04，B-lite 方案）。 配套文档： `04` （P1）、 `05` （P2）、 `06` （S6 调度插件）。开源贡献这件事本来就长在做项目的过程里——你在项目中踩过的坑、量出来的数据，最后都会自然地走到上游去。

## 1. 目标与验收（截止日驱动，不要拿 PR 数量自我感动）

贡献的动机保持不变：**获得第三方验证、积累面试素材、训练真实协作**（简历上多一行字只是副产品）。

| 死线 | 最低交付（信任线） | 冲刺交付 |
| --- | --- | --- |
| **2027.2 实习投递** | **在 Fluid 提至少 1 个带测量数据的 issue**（拿到维护者的正式回应或分类），外加 1 个流程级 PR（docs/test，已 merged 或进入 review） | Fluid 代码 PR 在途 |
| **2027.8 秋招** | **在 AIBrix 至少有 1 个代码级 merged PR**（或者拿到"维护者认可方向、PR 在 review"的完整证据链），两个社区合计至少 3 个 merged PR 或已回应 issue | AIBrix 至少 2 个代码 PR；与 Fluid 成员发生提名对话 |

**指标定义（v3 的核心修正）**：

- **信任线指的是在同一个社区内**从 docs/test 走到 code 的连续记录，它证明的事情是"维护者愿意再 review 你的东西"；**"横跨了 N 个社区"是一个虚荣指标**，与其在每个社区各撒一铲，不如在一个社区里攒出一条完整的信任线；
- 每一个 PR/issue 都必须能讲满一个完整故事：发现问题、定位根因、做出取舍、经历 review 攻防，讲不满的就不计入展示；
- typo 和格式类 PR 不拿来展示（但它可以作为你**第一次走流程**的合法动作，限 1 次）；被关掉的 PR 也不要藏着——"为什么被关、我从中学到了什么"同样是素材。

## 2. 社区情报档案（2026-10-05 现勘，入坑前请以仓库当日状态再复核一遍）

| 社区 | 仓库/许可 | 法律门槛 | 语言/渠道 | 入口标签 | 与我的关系与定位 |
|---|---|---|---|---|---|
| **Fluid** ⭐P1 主社区 | fluid-cloudnative/fluid，Apache-2.0，CNCF 沙箱 | **DCO**（`git commit -s`），没有 CLA | 中英文都可以，钉钉群很活跃；开发指南在 `docs/en/dev/how_to_develop.md` | issue 模板按类型分；有 PR 模板 | 与 P1 的闭环叙事同域（Dataset/DataLoad/调度注入）；**主回馈目标** |
| **AIBrix** ⭐P2 主社区 | vllm-project/aibrix，Apache-2.0 | DCO/CLA 以第一个 PR 时的 CI 机器人为准（vllm-project 系通常是 DCO），**阶段 0 核实** | 以英文为主，Slack 在 vllm-dev 的 `#AIBrix`；中文沟通也行得通 | `good first issue` / `help wanted` | P2 核心 2（冷启动协同）的回馈目标；**代码 PR 的主战场** |
| **JuiceFS CE** | juicedata/juicefs，Apache-2.0（旧的"AGPLv3"说法已经作废） | **必须签 CLA**（cla-assistant 会在第一个 PR 时引导你）；新源文件要加 license header；pre-commit 必装 | 中文完全可以，Discord/GitHub | `kind/good-first-issue` / `kind/help-wanted` | P1 的对照基线；**以使用和观察为主**，只接"使用过程中自然踩到"的 docs/小 bug PR；缓存组等企业版能力不向社区版提 |
| vLLM Production Stack | vllm-project/production-stack，Apache-2.0 | DCO | 英文 | good first issue | P2 的对照对象；与 Go router 同域，作为 AIBrix 之外的备选观察对象 |
| llm-d | CNCF 沙箱（2026 年新入） | DCO | 英文 | —— | P2 只读码观察，不投入贡献 |
| scheduler-plugins | kubernetes-sigs/scheduler-plugins，Apache-2.0 | CNCF DCO | 英文，K8s Slack | help wanted | **S6 的流程练习场**：用来练 fork/PR/CI，不背名额，也不与主社区抢时间 |
| Dragonfly | CNCF，Apache-2.0 | DCO | 中文可以 | good first issue | 只有当 P1 的 P2P 臂做深了才激活它，否则不碰 |
| vLLM 引擎本体 | vllm-project/vllm，Apache-2.0 | DCO | 英文 | good first issue | **不作为贡献目标**：Python/CUDA 域与我们的语言栈错位，我们只走读它的代码 |

**AIBrix 维护者的分工（@ 人的时候要按域来，别乱 @）**：routing/gateway 找 @varungup90、@xunzhuo；autoscaling 找 @nwangfw；KV offload/ 分离找 @DwyaneShi；benchmark/ 研究合作找 @happyandslow；control plane 稳定性找 @googs1025。成为组织成员的路径：先做出若干个贡献，再发 issue 申请加入 GitHub org。

**法律红线**：① 你自己的仓库（TrainCache/KVRouter）统一用 **Apache-2.0**；② 任何社区的代码你都可以读、可以学它的设计，但**绝对不能拷贝进自己的仓库**（即使同样是 Apache-2.0，你也要独立实现，并在 ADR 里注明"参考了 X 的思路"）；③ JuiceFS 企业版的代码和文档属于专有资产，你只能引用它的行为结论，禁止搬运；④ 第一次向任何社区提交之前，先把该仓库 CLA/DCO 的实际 CI 检查跑通一次（你可以故意在测试分支上看看红字长什么样，这不浪费时间）。

## 3. 三阶段检查点（与 04/05 的步骤精确耦合，不提前许愿）

| 检查点 | 时点（硬锚点） | 决定什么 | 依据 |
|---|---|---|---|
| **CP0 = 现在** | 本周（使用阶段启动） | stage0 立刻并行开跑，不等待任何可选项 | 纯学习动作，零沉没成本 |
| CP1 | P1 步骤 0 的现勘报告完成（约 2 周内） | Fluid 第一个 issue 的选题（文档不符/行为缺口/可观测性） | 现勘取证的结果，不靠想象 |
| CP2 | P1 步骤 7（核心 1 闭环）跑通之后 | Fluid 代码 PR 的选题与形态（先在 issue 里讨论） | 盈亏曲面里哪个发现对上游真的有用 |
| CP3 | P2 阶段 0 的现勘闸门关闭之后 | AIBrix 的第一个互动议题（冷启动测量 / 精准 vs 估算复现） | 头号风险（CPU KV 事件）的结论加读码笔记 |
| CP4 | P2 核心 2 主实验做完之后 | AIBrix 代码 PR（routing policy/选项/benchmark） | 旗舰图 2 的数据与维护者反馈 |
| 逃生舱 | 任意时点 | 降级为"只提 issue/数据，不写 PR"，简历叙事转回以项目为主 | 当流程长期受阻并且吞噬项目工期时启用 |

主社区是**被项目进度一步步推出来的**：Fluid 由 P1 的现勘与核心 1 推出，AIBrix 由 P2 的现勘与核心 2 推出。这里不存在"今天选错了社区"这回事，只存在"你还没有走到能拿出证据的那一步"。

## 4. 阶段 0：做一个观察者（本周启动，与双现勘合并执行）

**减负原则（v3 的关键修正）**：我们不在两个全家桶上各交一遍全量环境税。

### Fluid（全量做，约 3–4 个晚上）

1. `Watch` fluid-cloudnative/fluid（选 All activity）；只点 star 不算动作；
2. fork 加 clone，配好 upstream remote（官方 CONTRIBUTING 里的 5 步照做），跑通 `make test` 或 dev guide 指定命令下相关 Go 包的单测——Fluid 是纯 Go 仓库，你的笔记本完全能跑；跑不通的包要记下原

因（这很可能就是 issue 素材）；

3. **精读 10 个近期 merged 的 PR**，每篇做四行结构化笔记：`diff 规模 / commit 怎么切 / reviewer 追问什么 / 我能学到的形状`；
4. 精读 `CONTRIBUTING.md`、PR/issue 模板和 dev guide，记住几条硬规矩：DCO、PR 模板、每人最多 15 个 open PR、至少 1 个 approval 才会 merge；
5. 在钉钉群或 Slack 里潜水，记录社区的响应速度（这个数据会决定你在 CP1 时有多大信心）。

### AIBrix（轻量做，约 2–3 个晚上，拒绝全家桶黑洞）

1. `Watch` vllm-project/aibrix；加入 vllm-dev Slack 的 `#AIBrix`；
2. **只跑 quickstart**（helm 部署加一个示例请求打通），不要求你在本地编译 Envoy/Redis 全家桶，也不要求全测试变绿；
3. 读 `docs/` 里的贡献页与 `pkg/` 中 gateway/routing/autoscaling 相关的包，**只定向跑这些包的 Go 单测**；等第一个 PR 真的需要时，再补完整的开发环境；
4. 同样精读 **10 个 merged PR**（四行笔记），重点看 2026 年高频改动的 routing/connector/autoscaling 目录——这一步顺便完成了 P2 阶段 0 的读码任务，一份投入两份产出；
5. 核实 CI 中 DCO/CLA 的实际形态、PR 标题的惯例、lint 命令，记进笔记。

**stage0 的完成标准**：Fluid 相关包测试可跑 + 两个社区各 10 篇 PR 笔记 + AIBrix quickstart 截图 + 一页《社区地形笔记》（响应速度、常见 reviewer、PR 的典型形状、禁忌）。**时间盒为 1.5 周，一旦超时，优先保住 Fluid 全量与 AIBrix quickstart**。

## 5. 阶段 1：做一个 issue 报告者（P1/P2 现勘的顺产，零额外选题成本）

**现勘即贡献**——这是 v3 最重要的操作心法：你按 04/05 做现勘时产出的每一个"文档描述与实际行为不符"，都是一个低风险、高可信度的 issue。

候选选题（只能从证据里挑，不允许凭空想）：

- **Fluid 方向（CP1）**：① 现勘中实测到 webhook 注入的 nodeAffinity 行为与文档描述不一致，或者文档没有写清权重的覆盖条件 → 提 docs bug issue；② DataLoad 与调度时序上的可观测性缺口（没有任何事件或指标能表明预热进度）→ 提 feature proposal issue，**附上你的实测时序证据**；③ kind 或特定版本下部署文档里的坑（复现步骤要齐全）；
- **AIBrix 方向（CP3）**：① 冷启动测量话题——先开一个 discussion 或 issue，描述你的复现计划（扩缩容事件下 TTFT 凹陷的测量方法），问问社区是否已知、是否有相关 WIP；② 精准与估算两种模式在 CPU 后端的行为差异或文档空白；③ 你在跑 quickstart 和读文档时踩到的具体坑。

**纪律**：先搜一遍 issue/PR/discussion 的历史（连 closed 的也要搜）；中文社区可以用中文写，AIBrix 用英文（AI 只准用来润色文字，见 §9）；提 1–2 个就够，不要刷数。 **完成标准**：至少 1 个 issue 得到 maintainer 的正式回应（回复、分类、标 label 或转到 discussion）。

## 6. 阶段 2：流程练习 PR（每个社区最多 1 个）

- **Fluid**：把现勘中发现的文档错漏或过时部署说明做成一个 docs PR——**这是风险最低的第一个 PR**，完整走一遍流程：fork → branch → `git commit -s` → 按 PR 模板填全（动机/改动/验证）→ 过 CI → 回应 review；
- **AIBrix**：给你读过的 routing/autoscaling 包补一个**边缘场景测试**（比如副本重启或端点摘除之后的路由状态），选题之前先在对应 issue 下评论征得同意；
- 在 S6 的阶段 A/B 期间，scheduler-plugins 可以替代其中一次练习（K8s 机器人多、流程规范，很适合练习 E2E PR 礼仪），但它**不占主社区的名额**。

**定位纪律**：这两个 PR 的目标是"跑通流程，让 maintainer 见过你的 ID"，diff 控制在 200 行以内，不要多做。

## 7. 阶段 3：代码级 PR（从旗舰图里长出来，issue-first 是铁律）

### Fluid（CP2，2027.2 前后）

来源按可信度排序：

1. **自产数据（最强的故事）**：核心 1 盈亏曲面里发现的、对上游真实有用的点——候选形态包括：暴露 DataLoad 进度与就绪度的事件和指标、预热与调度协同的设计讨论（**先在 issue 里贴出你的测量数据与 proposal，major feature 按 Fluid 的惯例欢迎先写设计讨论**，等方向被认可之后再写代码）；
2. **认领 good-first/help-wanted**：在现勘期长期盯着这些标签，选中之后先评论一句"我想试这个，我的思路是 X，对吗"，等对方点头你再动手——这样既能防撞车，又能白嫖一次方案 review。

### AIBrix（CP4，2027 年夏天）

1. **自产需求**：核心 2 的冷启动协同——拿着 05 的事件研究数据开一个 issue（"scale-out 时新副本的 TTFT 惩罚测量，加上预热/加权放量的思路"），再按反馈决定落地形态（routing policy 选项、benchmark harness 增强，或者把行为文档化）；维护者可能会直接指给你一个已有 WIP——**被指向 WIP 一点都不丢人，这相当于一份免费情报，你要做的是立即调整选题**；
2. **认领**：从 `good first issue` / `help wanted` 里挑 routing、autoscaling、benchmark 域的条目，同样先评论认领。

**铁律**：直接甩过来一个大 PR，是新手的第一大死法；feature 类贡献一律先有 issue/discussion；500 行以内的小 PR，review 周期通常只有千行 PR 的零头，遇到大改动你要主动拆成系列 PR。 **完成标准**：至少 1 个代码级 merged PR，外加一份完整的故事复盘（问题怎么发现、证据是什么、方案上有什么分歧、最终做成什么形态、哪些部分没有被接受以及原因）。

## 8. issue 与 PR 模板

### 高质量 issue 骨架（中文/英文结构相同）

```
标题：[组件] 一句话现象（含关键报错或指标异常）
环境：版本（git commit/chart 版本）/ OS / kind vX / 关键配置（删掉敏感信息）
复现步骤（最小化，编号）：1. 2. 3.
期望行为 vs 实际行为：各一行
日志/数据：完整报错段或测量脚本+结果（代码块）
已排查项（区分新手与路人的关键）：
- 我确认过文档 X 节说 Y，与现象不符（附链接）
- 试过 A 配置，现象不变
- 怀疑与 B 有关，未验证（或：验证方法是 C，结果是 D）
```

反例（一股 AI 味的 issue）：没有版本号；复现步骤写着"按文档操作之后就报错"；没有已排查项；通篇都是礼貌的空话。**每一条技术陈述都必须是你亲手跑出来的**——maintainer 的第一个追问你要是答不上来，信用即刻归零。

### PR 描述结构（Fluid 用它仓库自带的 PR 模板；AIBrix 按贡献页的六要点写）

1. 动机与关联 issue（`Fixes #n` 或 `Related to #n`）；2. 改动点列表；3. **如何验证**（命令加结果，这是 maintainer 最看重的段落）；4. 测试（新增单测或边缘用例）；5. 文档（行为发生变化时必须同步更新 docs）；6. commit 标题不超过 50 个字符，一个 commit 只做一件改动。

## 9. 与 AI 协作的红线（可操作版）

- **事实层你自己写**：环境、复现步骤、日志、测量结果——AI 根本不知道你的机器上发生了什么；
- **动机层你自己打底**：为什么改、方案上有什么取舍，先用中文写出草稿，再让 AI 帮你润成英文；
- **diff 你必须自己吃透**：reviewer 有权问"这行为什么这么写"，你答不上来就等于信用破产；
- **去 AI 味**：交稿之前删掉空洞的总结句，保证每一段要么有数据要么有步骤；礼貌要保留，但"thanks for your great work"式的填充客套要删掉；
- vLLM 系文档已经有 "AI Assisted Contributions" 政策，第一个 PR 之前去读一下当期版本，按它的披露要求做。

## 10. PR 全流程的工程细节（分社区列出，帮你少死一次）

| 事项 | Fluid | JuiceFS | AIBrix / Production Stack |
| --- | --- | --- | --- |
| 签名 | DCO `git commit -s`（漏签了用 `git commit --amend -s`） | **CLA 机器人**在第一个 PR 时引导；有没有 DCO 以 CI 为准 | 第一个 PR 之前核实 DCO 机器人 |

| 事项 | Fluid | JuiceFS | AIBrix / Production Stack |
| --- | --- | --- | --- |
| 风格 | Go 加仓库 lint | Effective Go 加 go fmt 加 pre-commit，新文件要 license header | 仓库 lint/pre-commit；Go/Python 分包 |
| commit/标题 | 祈使句，具体惯例读 10 篇 PR 自己总结 | 解释性 message，不同逻辑单元分开 | 标题不超过 50 字符；遵循仓库的前缀惯例 |
| review 纪律 | 1 个 approval；**open PR 不超过 15 个** | PR 必须关联 issue；大特性先写设计文档 | **review 期间避免不必要的 force-push**（官方明写，用追加 commit，merge 时由维护者处理历史） |
| 渠道 | 钉钉群里 7 天没有 review 就礼貌问一次 | GitHub/Discord | Slack `#AIBrix`，按域 @ 维护者 |

通用规矩：review 意见要在 **48 小时内响应**；每次 push 都在 PR 里留言说明改了什么（reviewer 不想自己去找 diff）；遇到不合理的意见，就摆数据讨论，既不硬顶也不无脑照改；方向性分歧被否决时，体面收尾（"Thanks, I see the rationale, closing this one."）——**体面退场同样是在积累社区信用**。

## 11. 反模式（死亡方式清单，每周自检一次）

1. 不搜历史就开 issue/PR（重复提案，信用一开场就是负的）；
2. "我先把大 PR 写完再来和你们说"（这种 PR 必关）；
3. 认领之后玩消失、占坑不更新（超过 2 周没有进展就要主动留言说明）；
4. 为了凑数提 typo/格式 PR（它在信任线里属于噪声，只允许在走流程时用 1 次）；
5. 本地测试不跑、CI 红了就等着机器人来教（CI 红灯等于在给维护者发"我需要保姆"的信号）；
6. 在 PR 里夹带无关的重构（一个 PR 只做一件事）；
7. 催 review 时带上情绪，或者被拒之后删叉走人；
8. 让贡献倒灌项目：为了凑 PR 去改变 P1/P2 的研究方向——**这个关系永远是项目产生贡献，贡献不能反过来牵引项目**；
9. 在两个社区同时开半成品的坑（精力一旦碎片化，两条信任线都建立不起来）。

## 12. 风险与对策

| 风险 | 对策 |
| --- | --- |
| issue 放了 7 天没人理 | 礼貌地 bump 一次；还是没有回应就换选题，不要恋战；中文渠道可以在群里贴 issue 链接 |
| good first issue 被别人抢了 | 先评论认领并说出思路；常备 2–3 个备选 |
| AIBrix 迭代太快、PR 频繁冲突 | 遵守 rebase 纪律：每周 rebase 一次 main；冲突在 24 小时内处理；选题贴近当周正在改的文件，反而更容易被 review |

| 风险 | 对策 |
| --- | --- |
| Fluid 的环境或测试在本机跑不通 | 这本身就是一个合法的 issue；卡住 3 天就在群里问，问题要带上版本号和报错，不要问"怎么入门" |
| review 周期以月计 | 这很正常，你并行推进项目就好，别让它阻塞死线；每个月礼貌问一次 |
| 现勘结论推翻了缺口（Fluid 已经做了块级门控） | 按 04 的闸门收缩叙事：贡献改为提供复现数据或做文档澄清，不要硬推 feature |
| 贡献吞噬项目工期 | 遵守每周开源预算的硬上限（见 §13）；超了就先保项目；逃生舱随时可用 |
| 法律条款发生变化（DCO/CLA/许可证） | 每次提第一个 PR 之前，都以仓库当日的 CONTRIBUTING/LICENSE 为准，笔记里记下复核日期 |

## 13. 周节奏与本周行动清单

**常态预算**：使用阶段每周不超过 5 小时（stage0 期间可以到 7 小时）；项目主线期每周不超过 3 小时，只做"回 issue、改 PR、读 PR"这类碎片动作；自产 PR 的集中写作，放在旗舰图完成之后的整块周末。

**本周（stage0，CP0）逐日安排**：

1. D1：Watch fluid-cloudnative/fluid 与 vllm-project/aibrix；加入 Slack 的 `#AIBrix`；fork 两个仓库并配好 upstream；
2. D2：在 Fluid 仓跑 `make test` /dev guide 的首步，记录环境问题；
3. D3：用 helm 部署 AIBrix quickstart 并打通一个示例请求，截图归档；
4. D4–5：每个仓库精读 5 篇 merged PR（四行笔记，共 10 篇）；
5. D6：Fluid 再读 5 篇 PR，并定向跑 P1 同域包（dataload/scheduling 相关）的单测；在 AIBrix 跑通 routing 相关 Go 包的单测；
6. D7：写一页《社区地形笔记》（包含 DCO/CLA 实际 CI 的截图、响应速度初判、禁忌清单），作为 CP1 的输入。

## 14. 诚实清单（面试时主动说）

- 我的贡献是"项目驱动"的：issue 和 PR 都长自 P1/P2 的现勘与测量，我没有为了贡献而贡献；
- typo PR 只有一个，它的作用是帮我走通流程，我不会把它包装成"活跃贡献者"；
- 被关闭或方向被否的 PR 我会保留下来，并且讲得清楚原因；
- 跨社区我只有 Fluid 与 AIBrix 两条信任线，scheduler-plugins 只是练习场；
- 我没有碰 vLLM 引擎内核与 JuiceFS 企业版：它们落在我的语言栈和许可证边界之外，我选择用读码和调研来覆盖这部分知识，不会夸大。

## 15. v2 → v3 变更摘要

1. 主社区：从 JuiceFS/AIBrix 双主改为 **Fluid/AIBrix 双主**，JuiceFS 降为使用基线与观察社区（缓存组是企业版能力，社区版没有提 PR 的空间）；
2. 指标：从"横跨至少 2 个社区"改为**同社区信任线**；scheduler-plugins 明确定位为 S6 练习场；
3. stage0：从两个仓库全量环境改为 Fluid 全量、AIBrix quickstart 减负；PR 笔记从每仓 20 篇降为 10 篇；
4. 事实更正：JuiceFS CE 是 Apache-2.0（但需要签 CLA 并加 license header）；Fluid 是 CNCF 沙箱、要 DCO、有 PR 模板、15 个 open PR 上限；AIBrix 的维护者分工与 force-push 纪律记入档案；
5. 检查点与 04/05 的步骤精确耦合（CP1–CP4），死线锚定在 2027.2 与 2027.8；
6. 新增内容：社区情报档案、从现勘和旗舰图生长出来的具体选题候选、分社区工程细节表、九条反模式、周节奏与本周逐日清单。